LAB 5

Build, Test, and Package Applications with IntelliJ IDEA and Docker

Student Name: Sidra-Tul-Muntaha

Registration Number: 23-BSCS-84

Course:DevOps

GitHub Username:sidra2244

Task 1 — Clone and Launch IntelliJ IDEA

Purpose

Clone the supplied IntelliJ Docker repository, launch the browser desktop, and open IntelliJ IDEA.

Screenshot Evidence

Screenshot: intellij_repo_clone.png

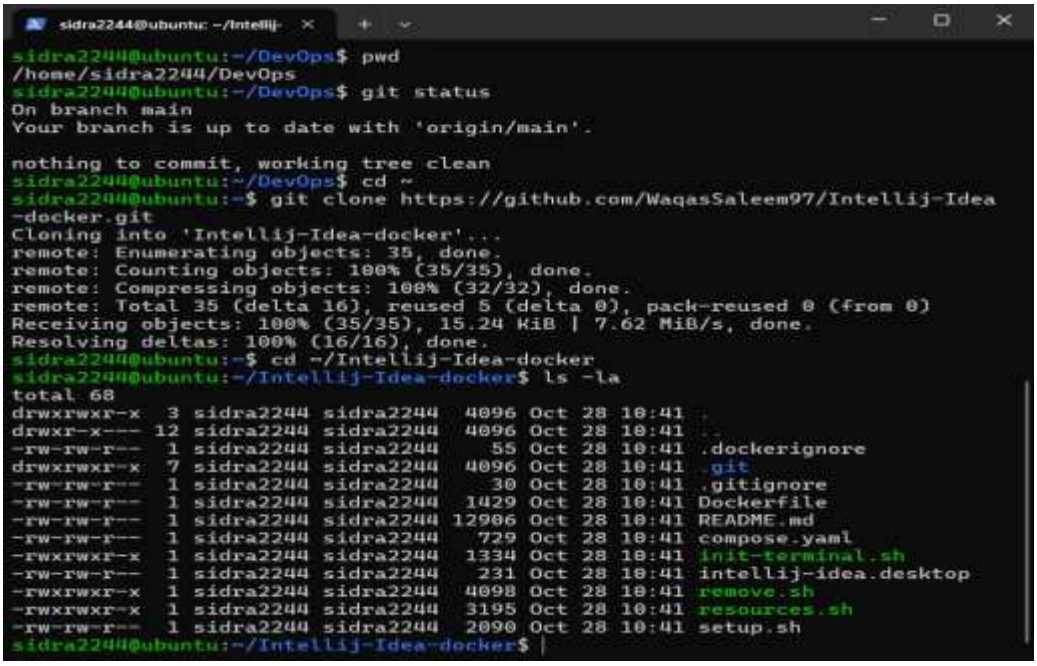


Figure caption: Repository clone command and successful download.

Screenshot: intellij_running.png

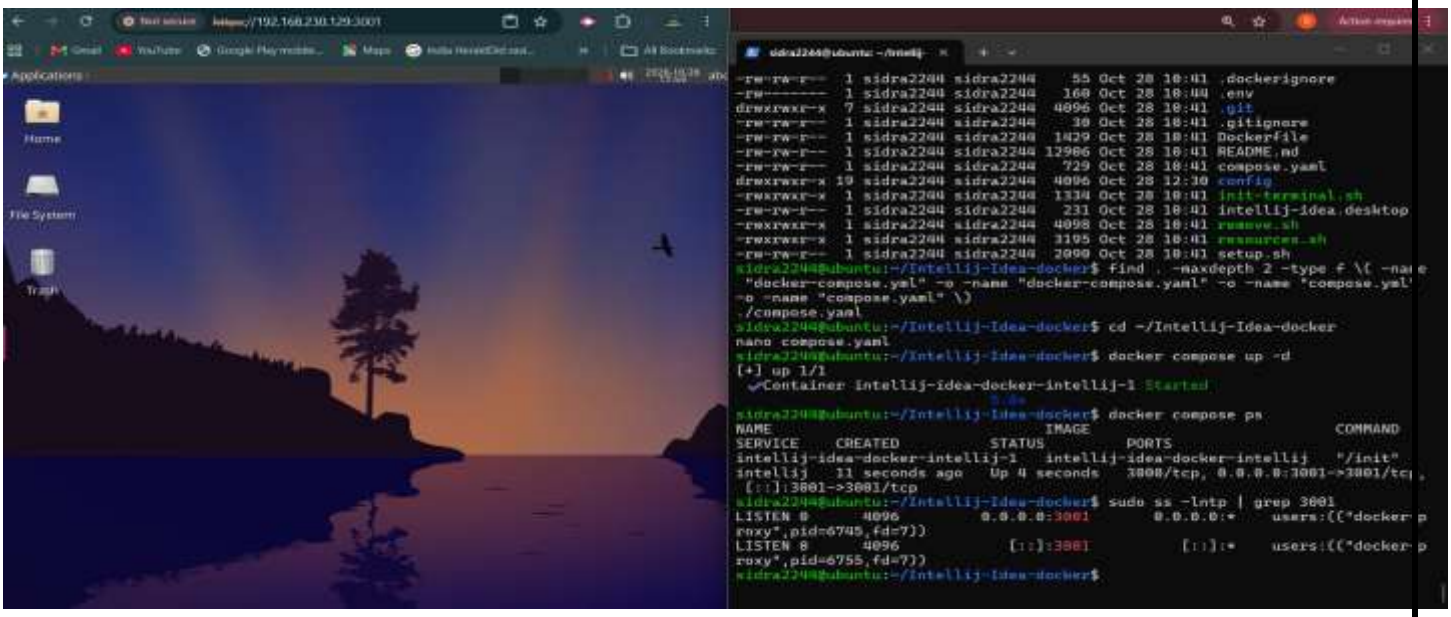


Figure caption: Running intellij service and desktop port 3001.

Screenshot: intellij_desktop.png

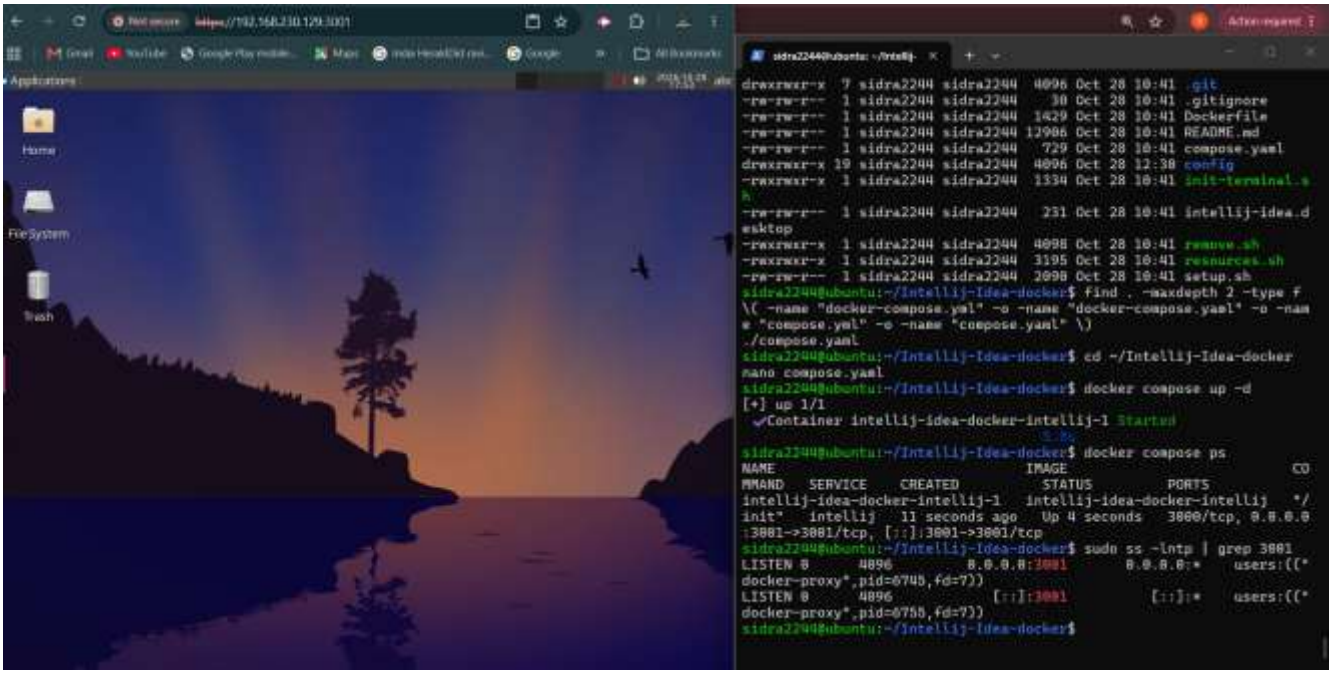


Figure caption: IntelliJ IDEA open in the browser desktop, with identity evidence visible.

Task 2 — Build and Run the Maven Project

Purpose

Clone the Maven sample, install Maven, add Elasticsearch 7.0.0, package and test the project, then run its JAR.

Screenshot Evidence

Screenshot: maven_project_clone.png

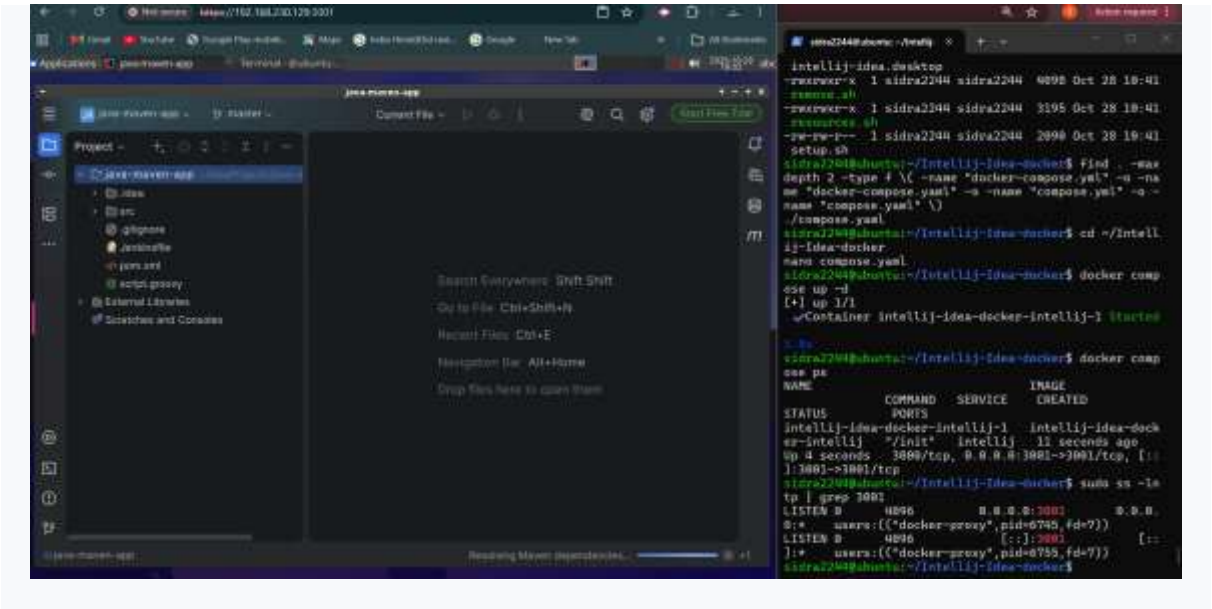


Figure caption: Maven repository URL and clone destination.

Screenshot: maven_installed.png

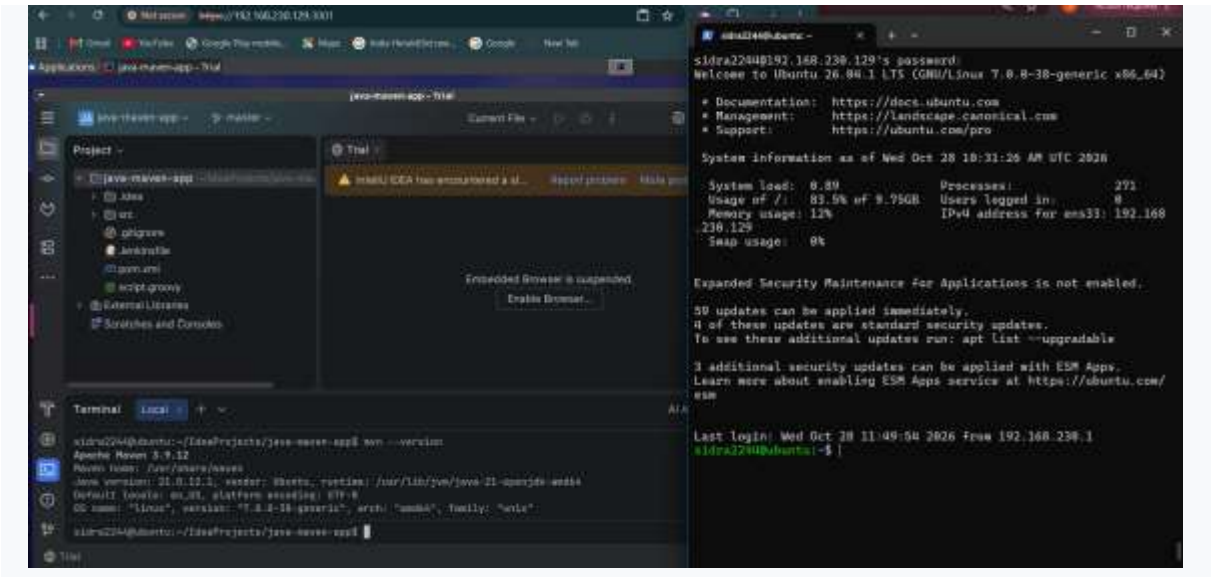


Figure caption: Installed Maven version.

Screenshot: maven_package.png

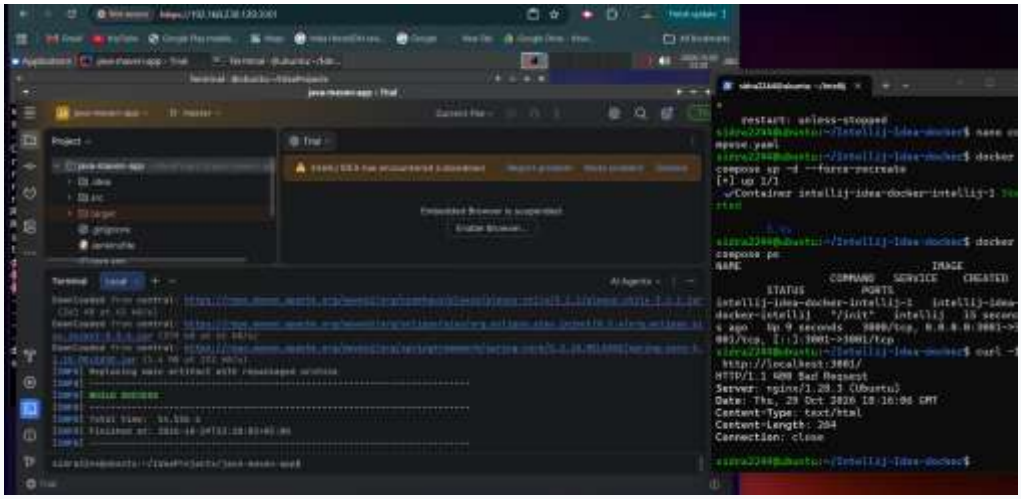


Figure caption: Initial Maven package command and BUILD SUCCESS.

Screenshot: maven_elasticsearch_dependency.png

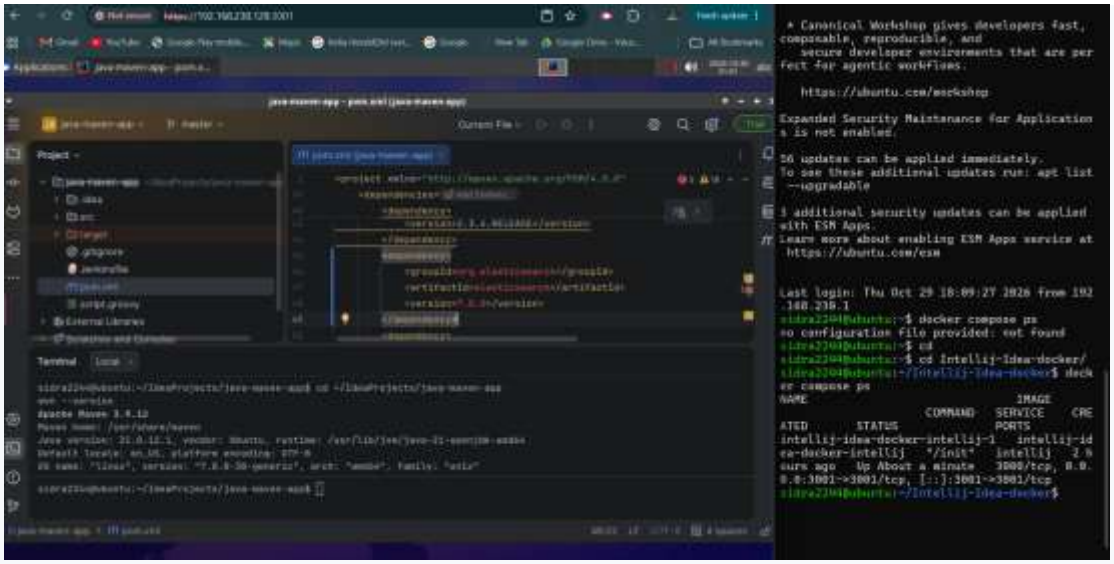


Figure caption: Elasticsearch 7.0.0 dependency and synchronized Maven entry.

Screenshot: maven_repackage.png

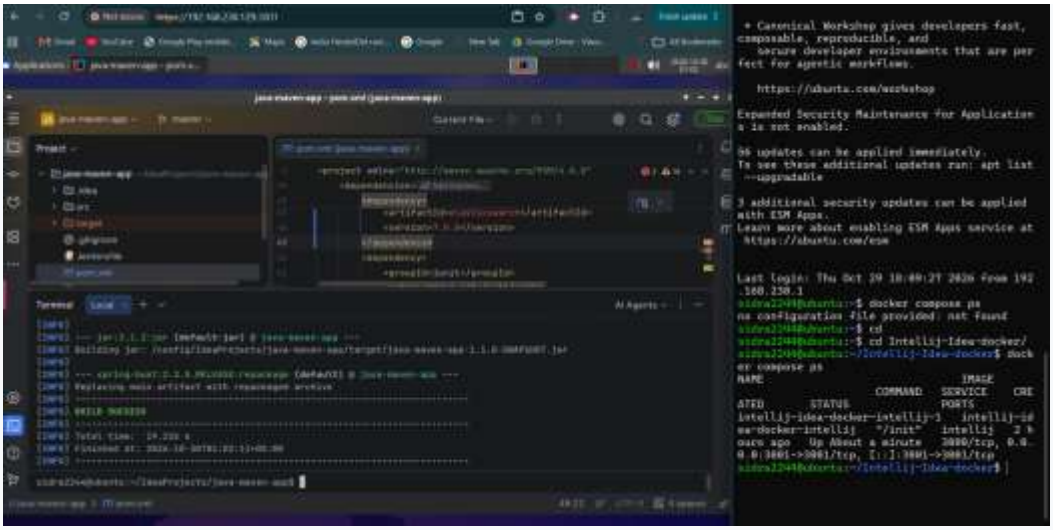


Figure caption: Successful package build after the dependency change.

Screenshot: maven_test.png

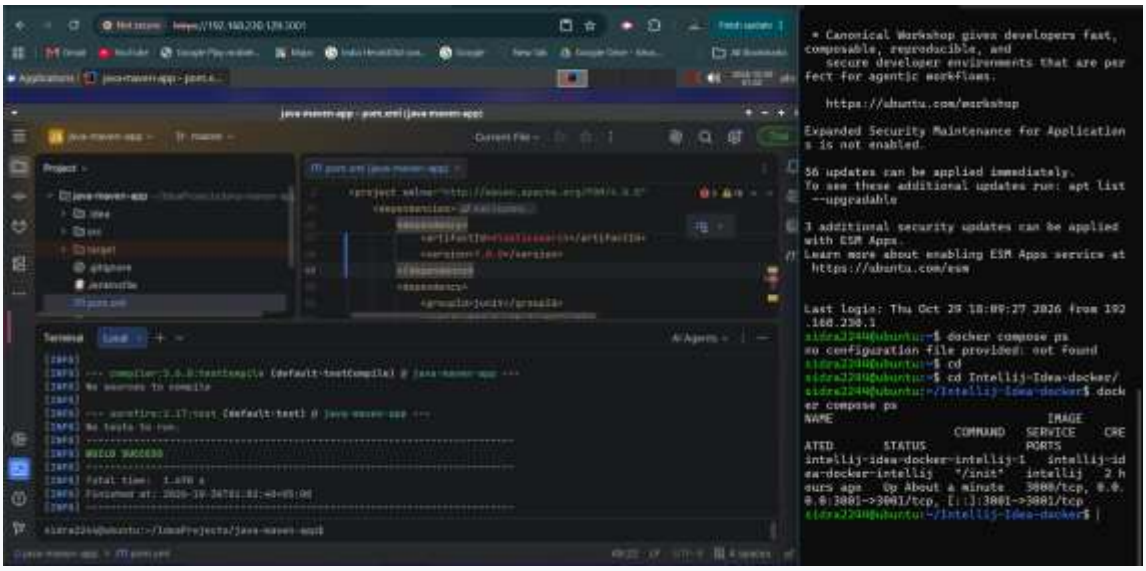


Figure caption: Test phase output and build result.

Screenshot: maven_jar.png

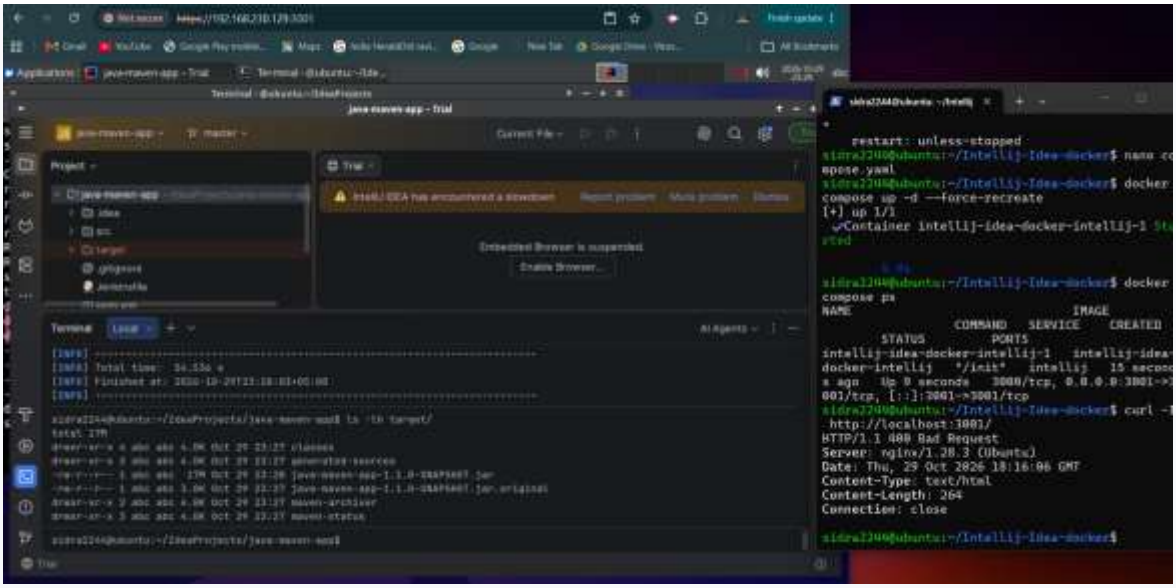


Figure caption: Exact JAR filename under target.

Screenshot: maven_running.png

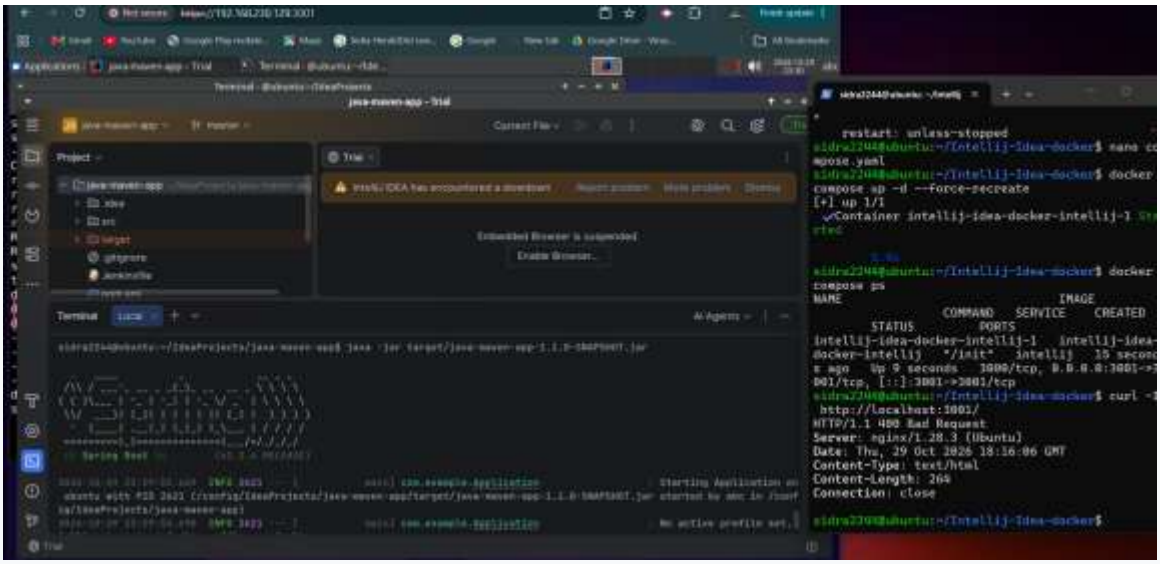


Figure caption: JAR command and successful application startup.

Screenshot: maven_stopped.png

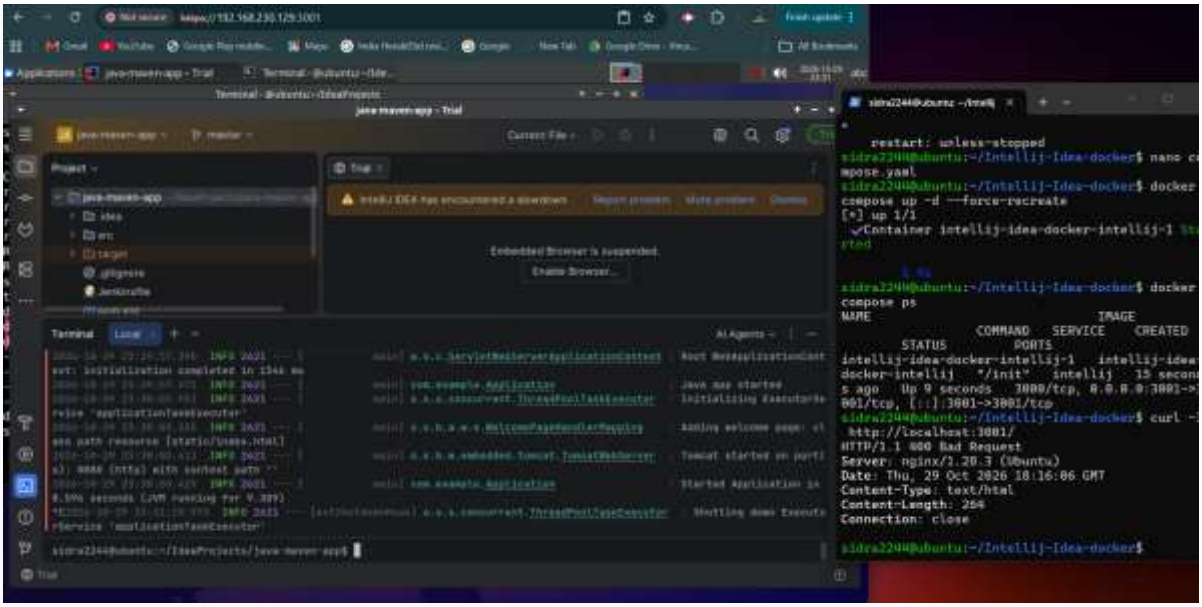


Figure caption: Application stopped with Ctrl+C and prompt returned.

Screenshot: maven_project_closed.png

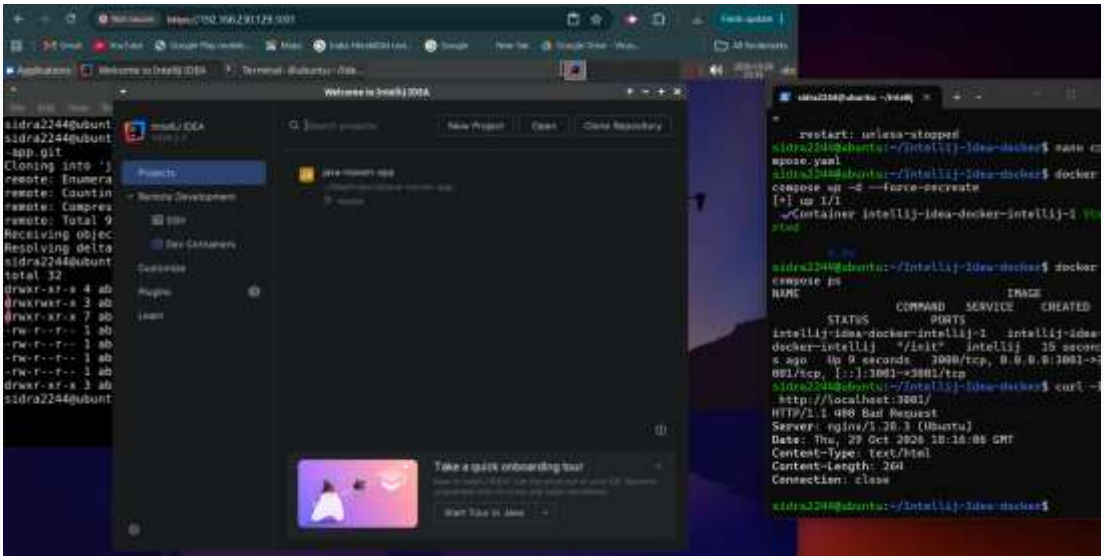


Figure caption: IntelliJ Welcome screen after closing the Maven project.

Task 3 — Build and Run the Gradle Project

Purpose

Build the Gradle sample, add Elasticsearch 7.0.0, rebuild it, and run the generated JAR.

Screenshot Evidence

Screenshot: gradle_project_clone.png

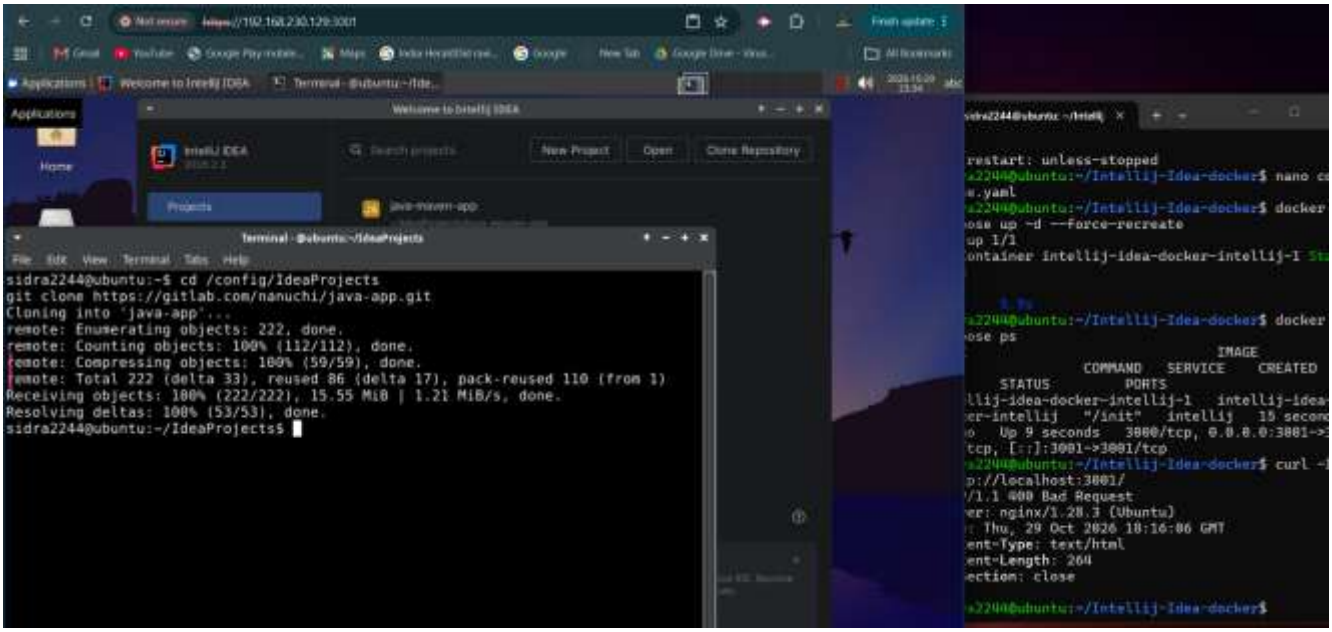


Figure caption: Gradle repository URL and clone destination.

Screenshot: gradle_build.png

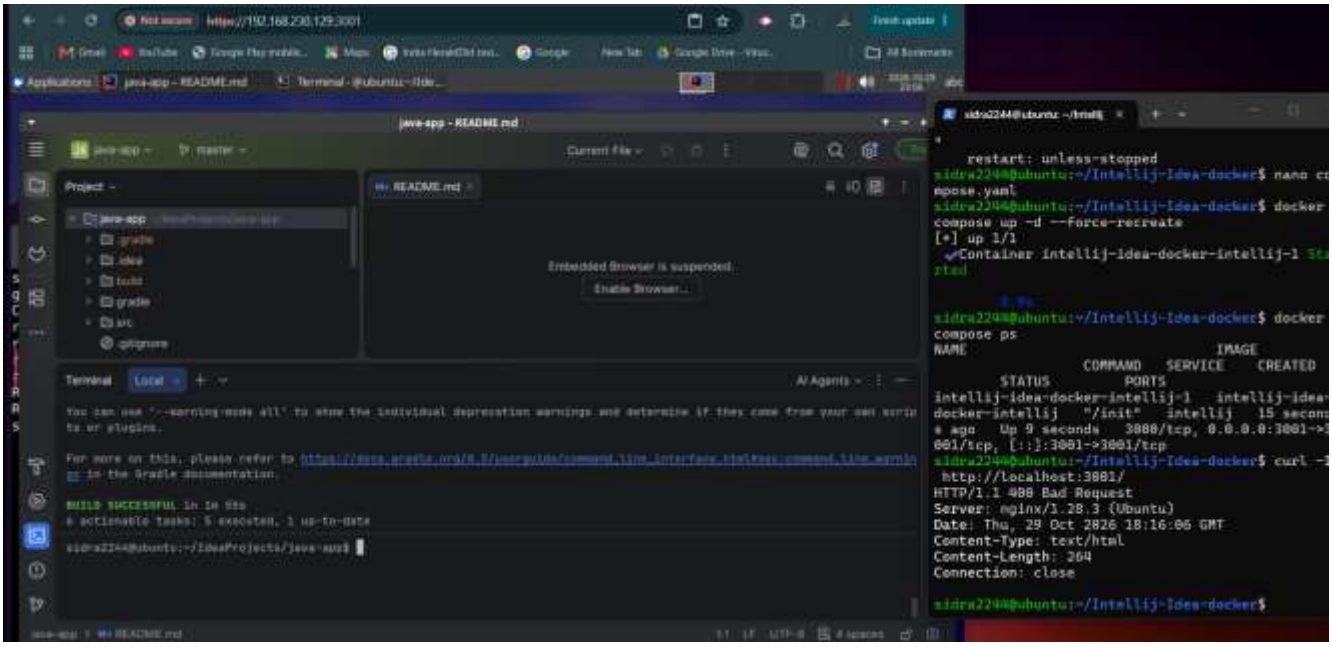


Figure caption: Initial successful Gradle build.

Screenshot: gradle_elasticsearch_dependency.png

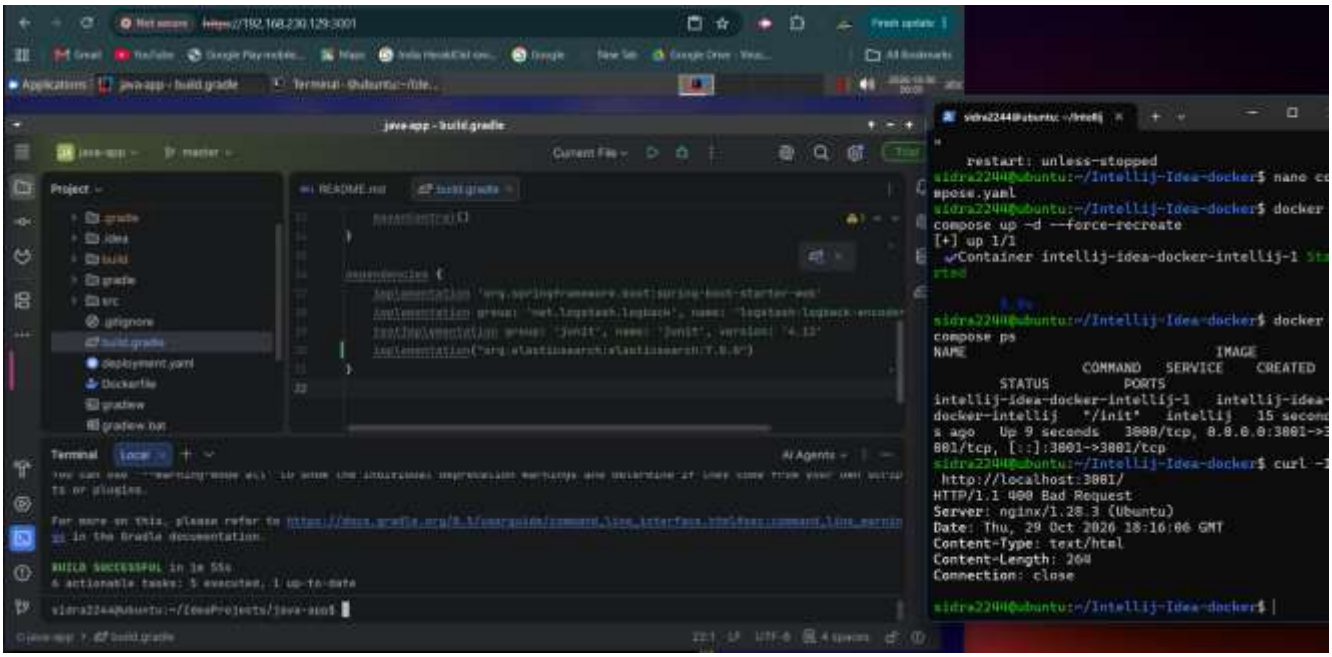


Figure caption: Dependency declaration and synchronized library.

Screenshot: gradle_rebuild.png

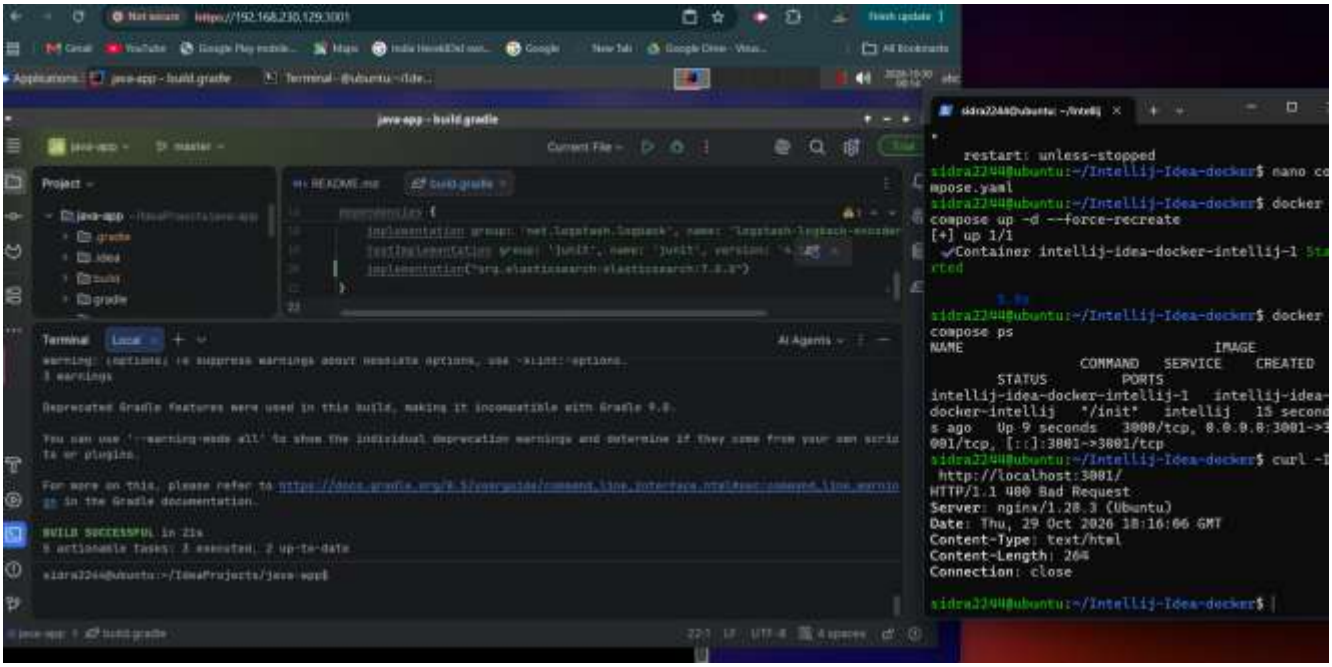


Figure caption: Successful rebuild after adding Elasticsearch.

Screenshot: gradle_jar.png

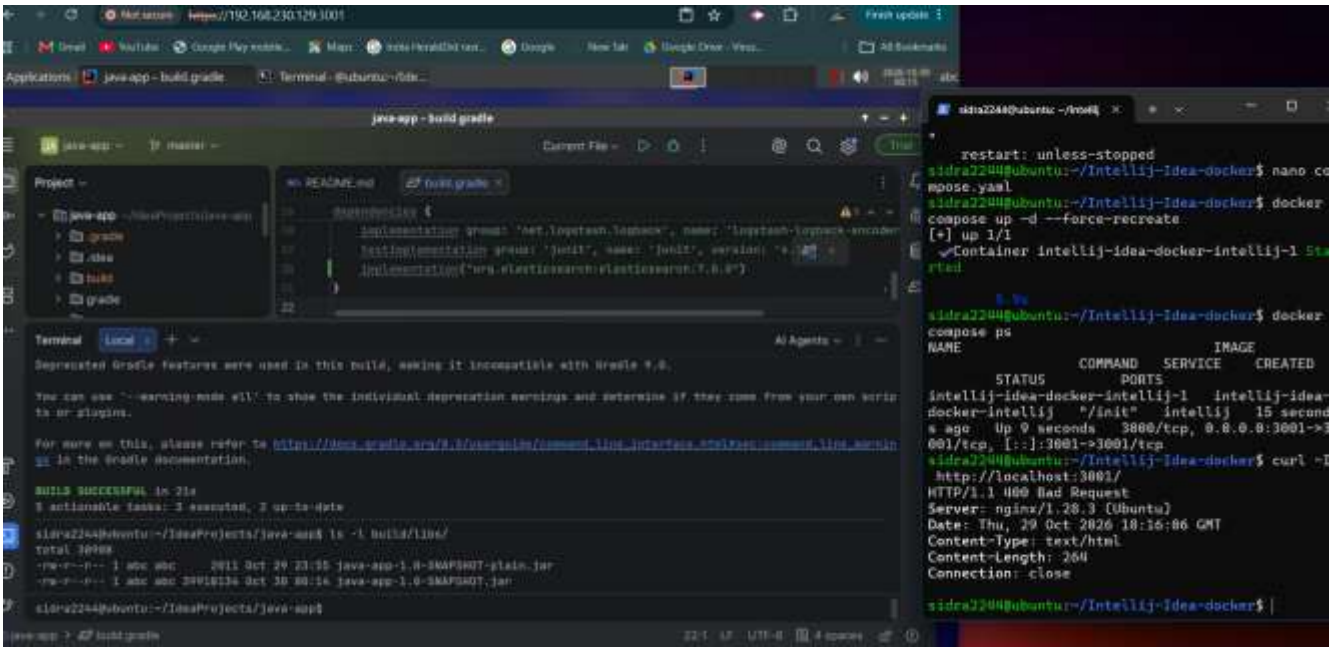


Figure caption: Exact JAR filename under build/libs.

Screenshot: gradle_running.png

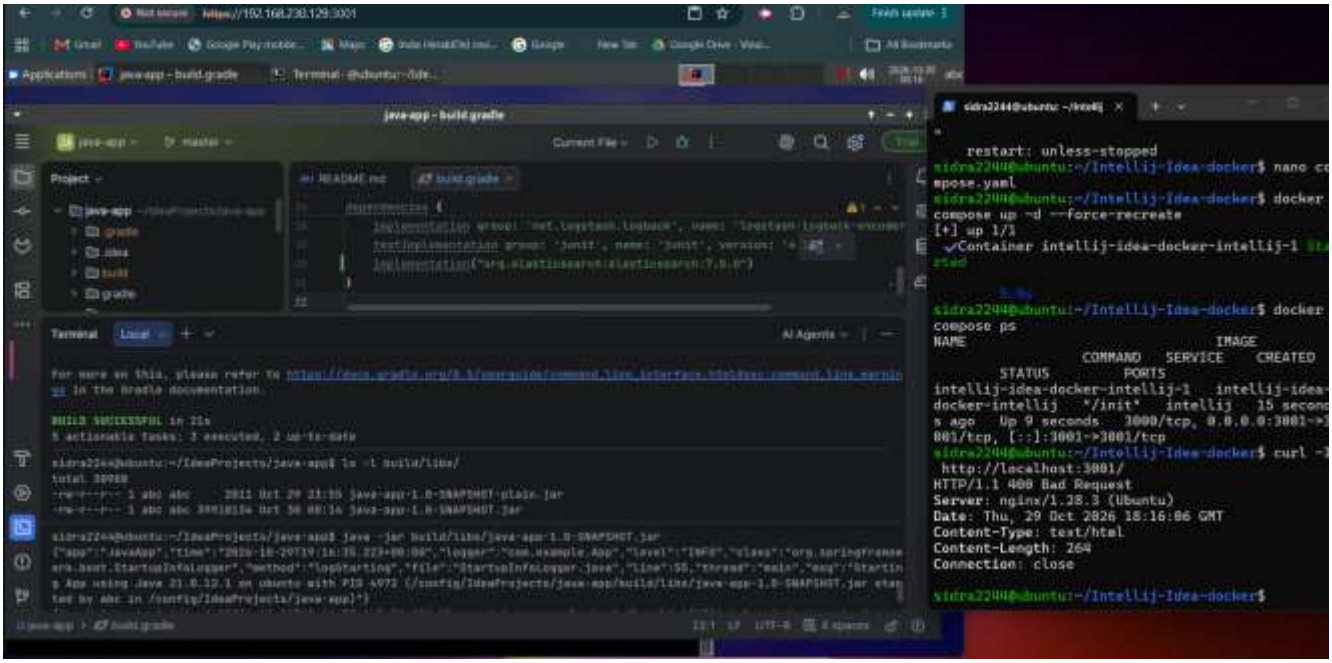


Figure caption: JAR command and application startup.

Screenshot: gradle_stopped.png

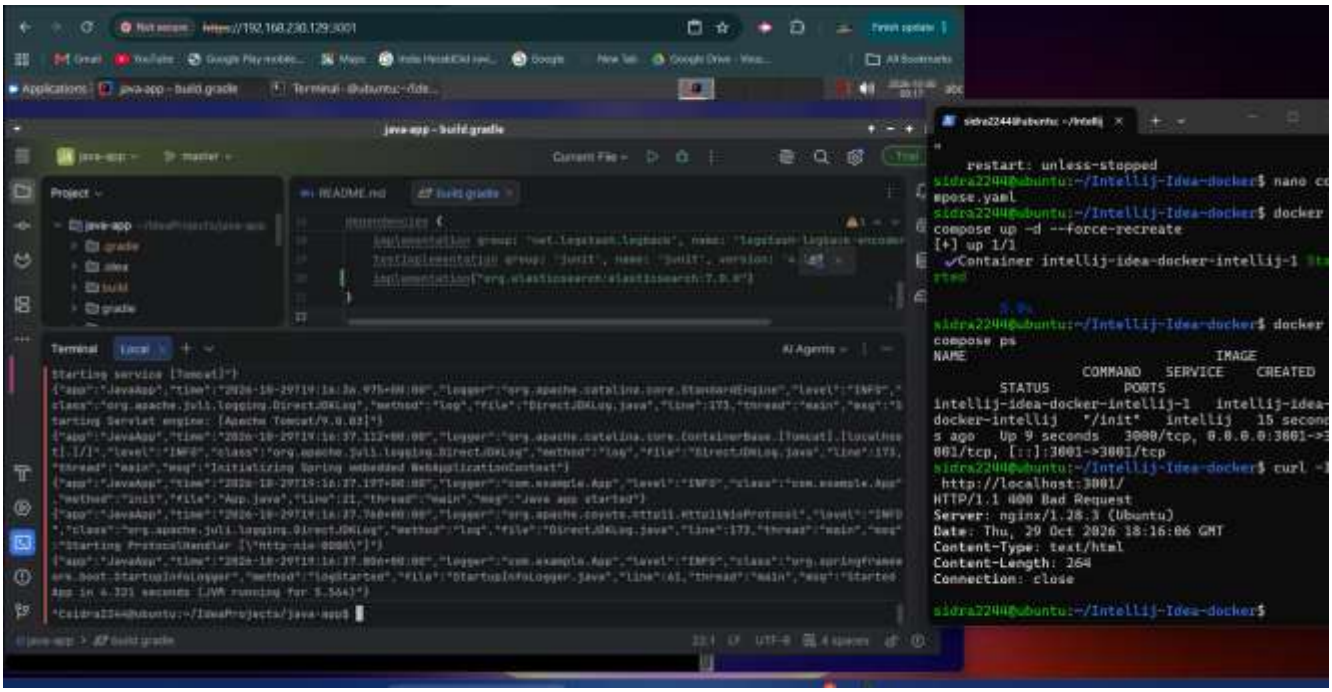


Figure caption: Application stopped and terminal prompt returned.

Screenshot: gradle_project_closed.png

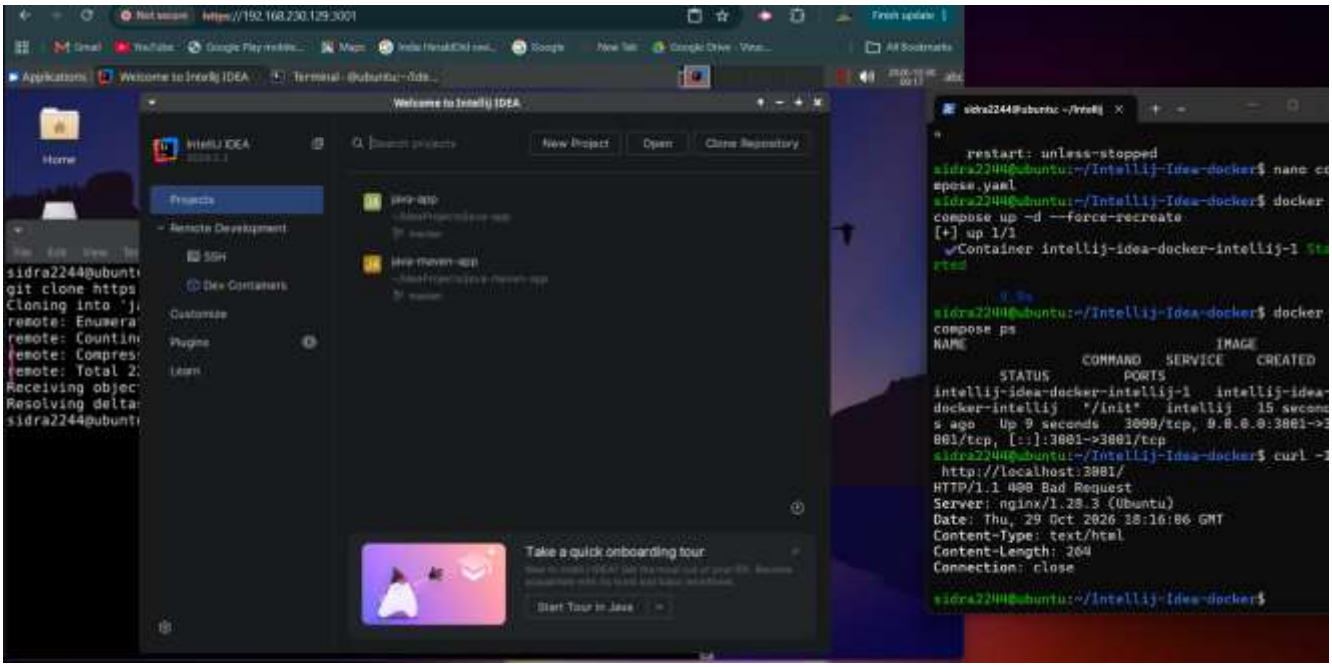


Figure caption: IntelliJ Welcome screen after closing the Gradle project.

Task 4 — Install and Run the Node.js Project

Purpose

Install Node.js and npm, run the sample application and tests, and create an npm package.

Screenshot Evidence

Screenshot: node_project_clone.png

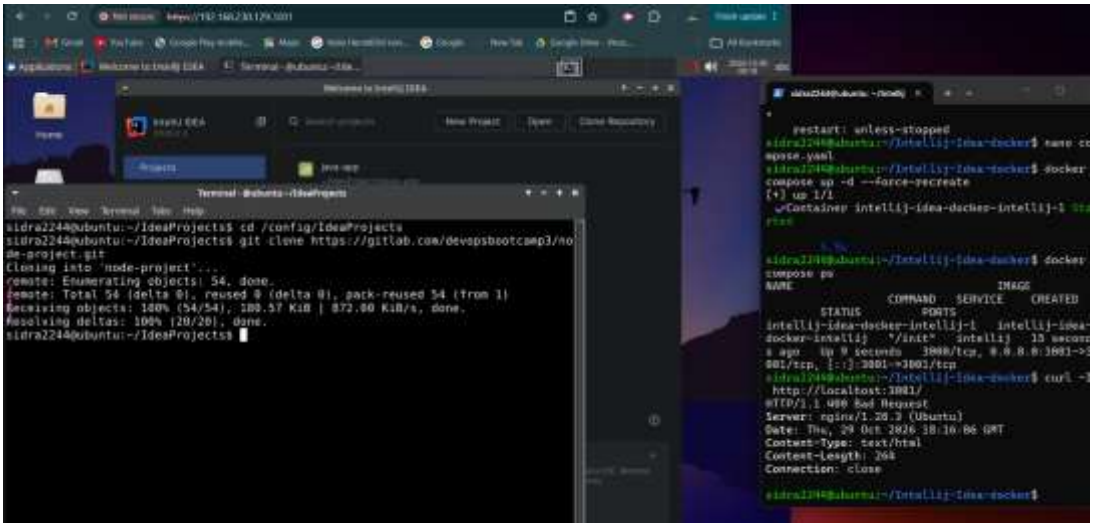


Figure caption: Node.js repository URL and clone destination.

Screenshot: nodejs_installed.png

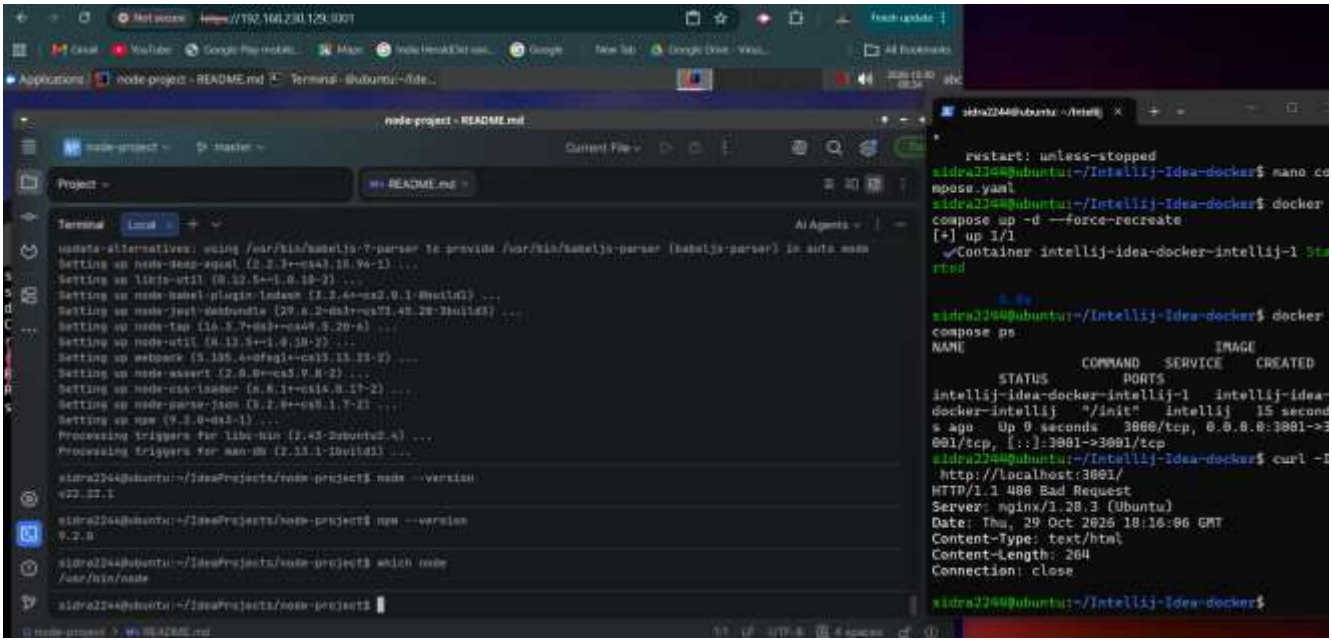


Figure caption: Installed Node.js and npm versions.

Screenshot: npm_install.png

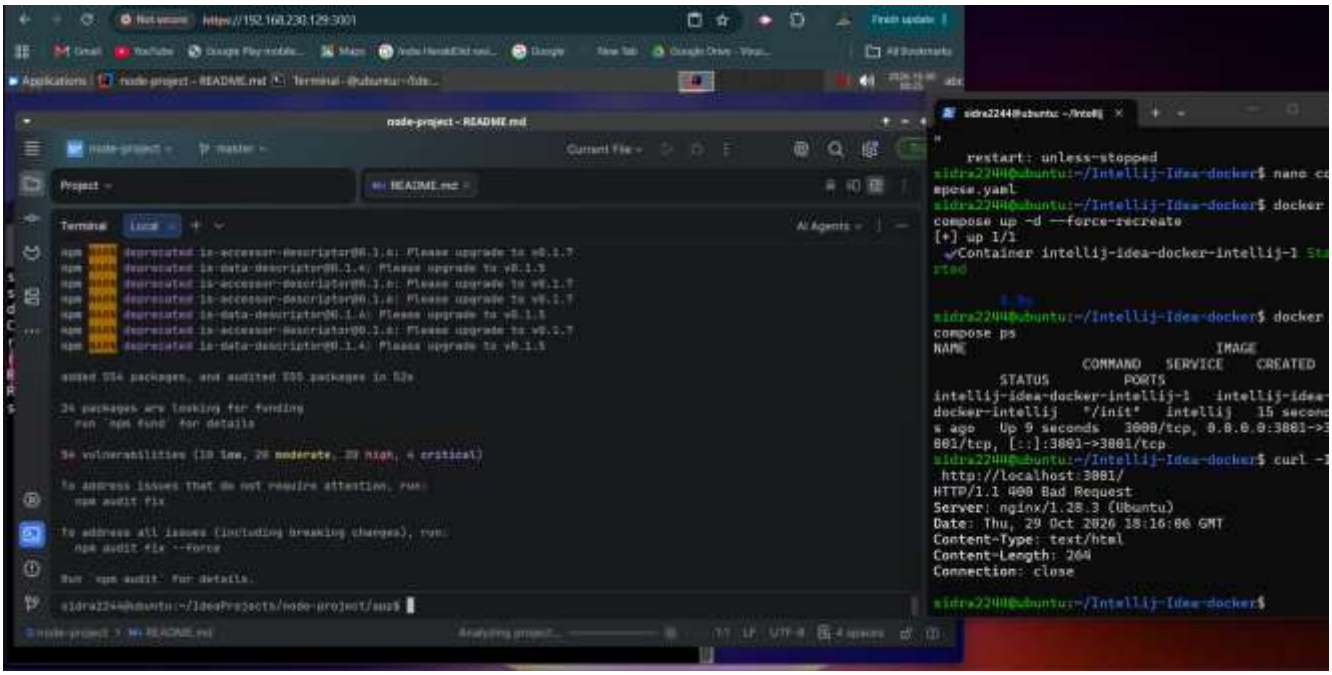


Figure caption: Completed npm dependency installation.

Screenshot: node_running.png

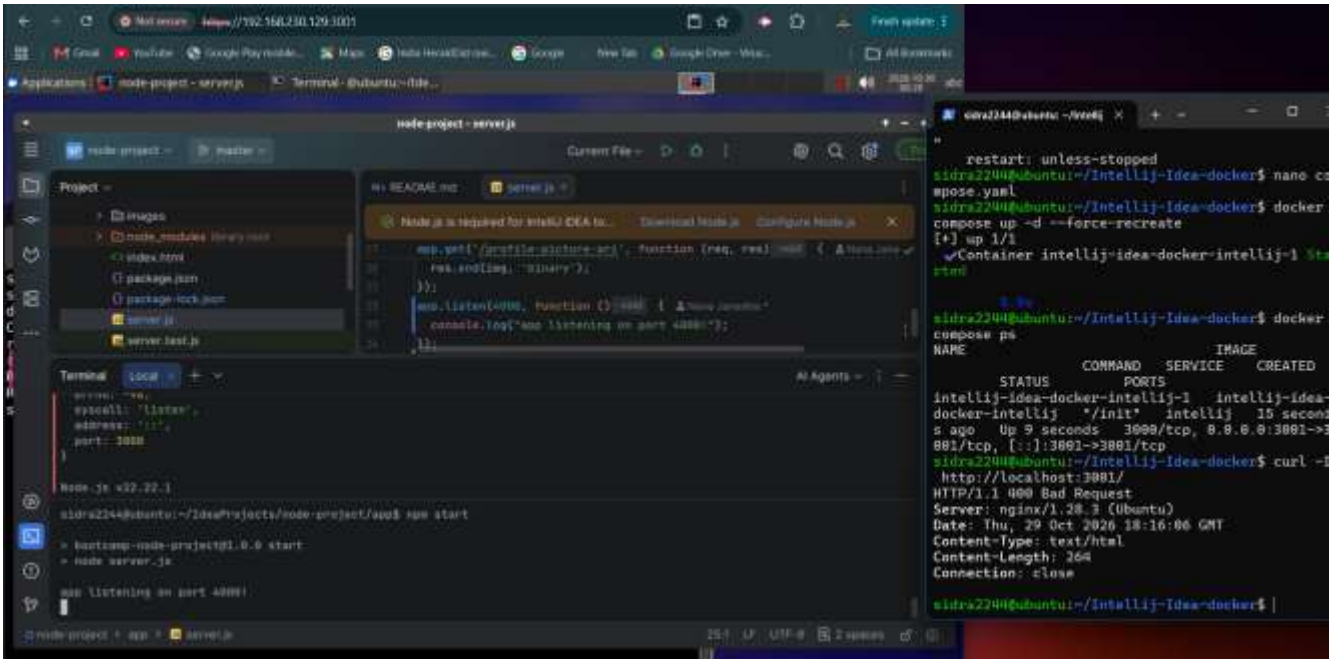


Figure caption: Application listening on port 4000.

Screenshot: node_browser.png

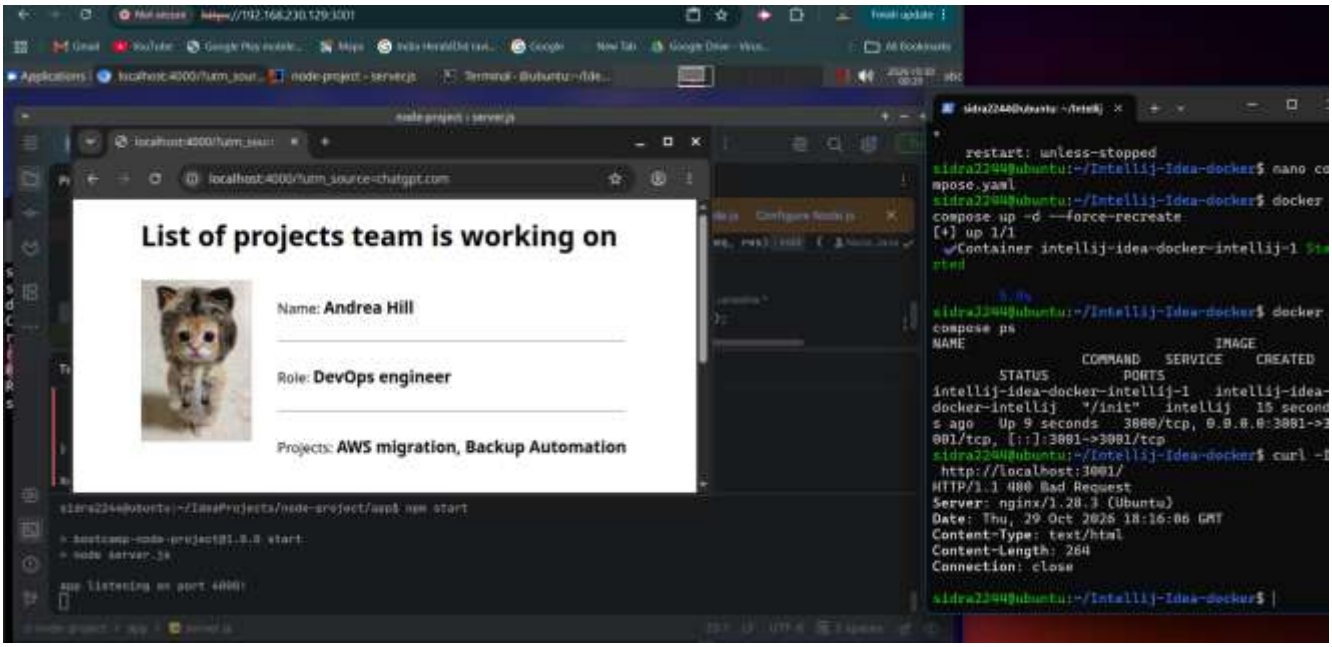


Figure caption: Application page at localhost:4000 and IntelliJ terminal.

Screenshot: npm_test.png

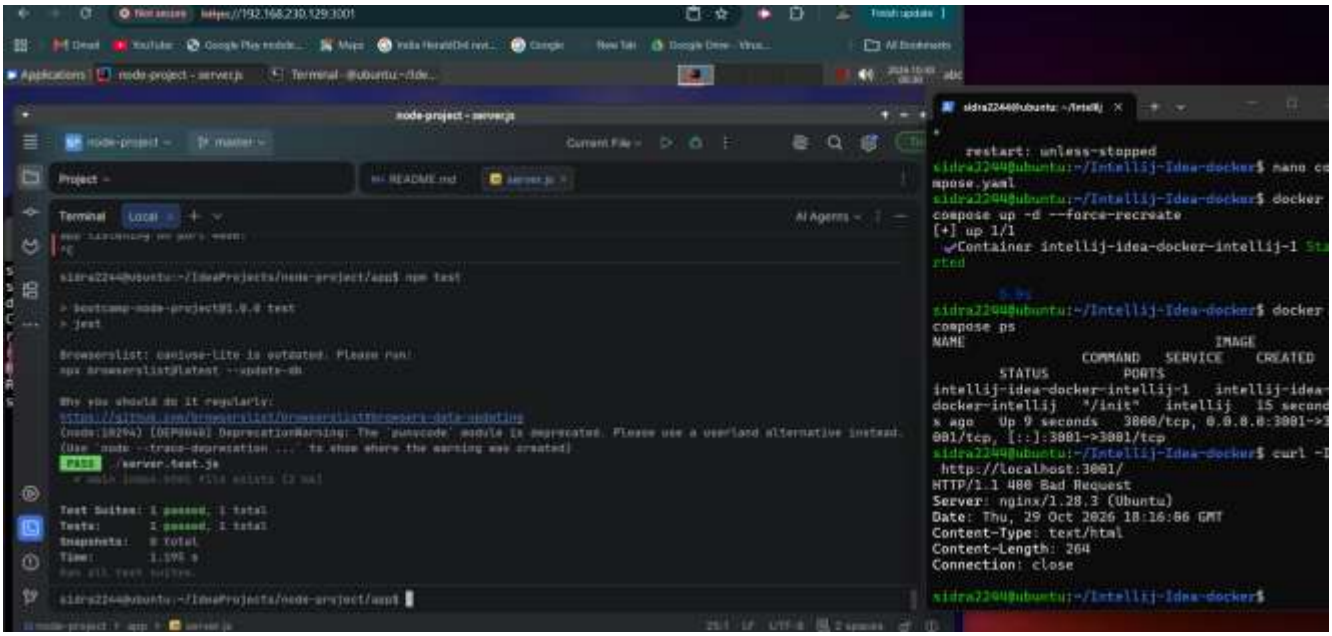


Figure caption: Passing Jest test summary.

Screenshot: npm_pack.png

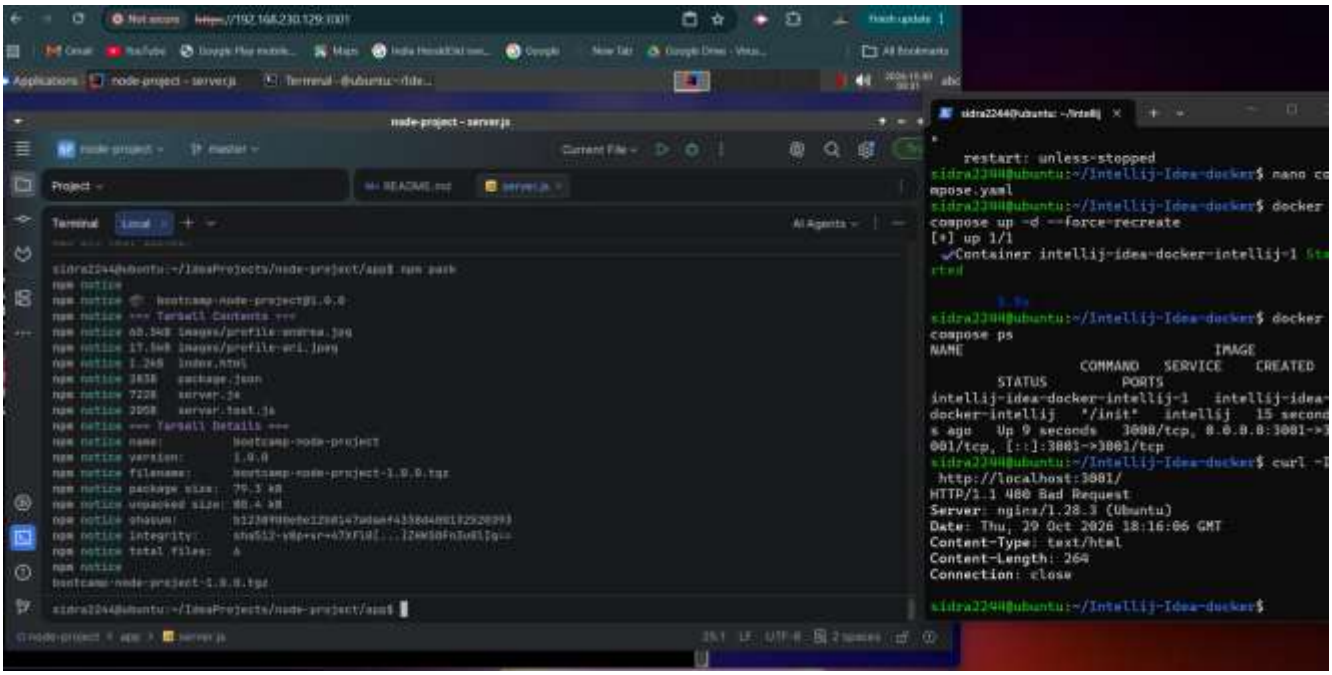


Figure caption: Generated bootcamp-node-project-1.0.0.tgz package.

Task 5 — Build the React and Node.js API Artifact

Purpose

Install API dependencies, build the Node.js API bundle with webpack, and inspect the generated bundle.

Screenshot Evidence

Screenshot: react_node_project_clone.png

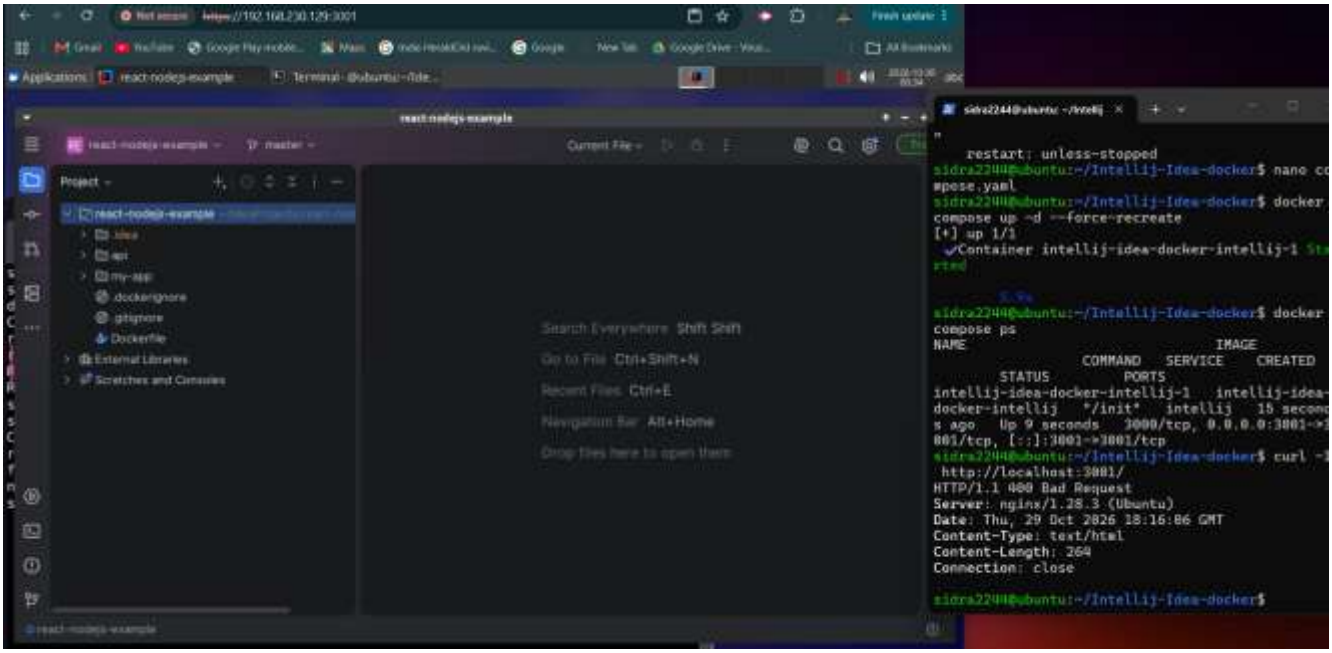


Figure caption: React/Node.js repository URL and clone destination.

Screenshot: react_api_dependencies.png

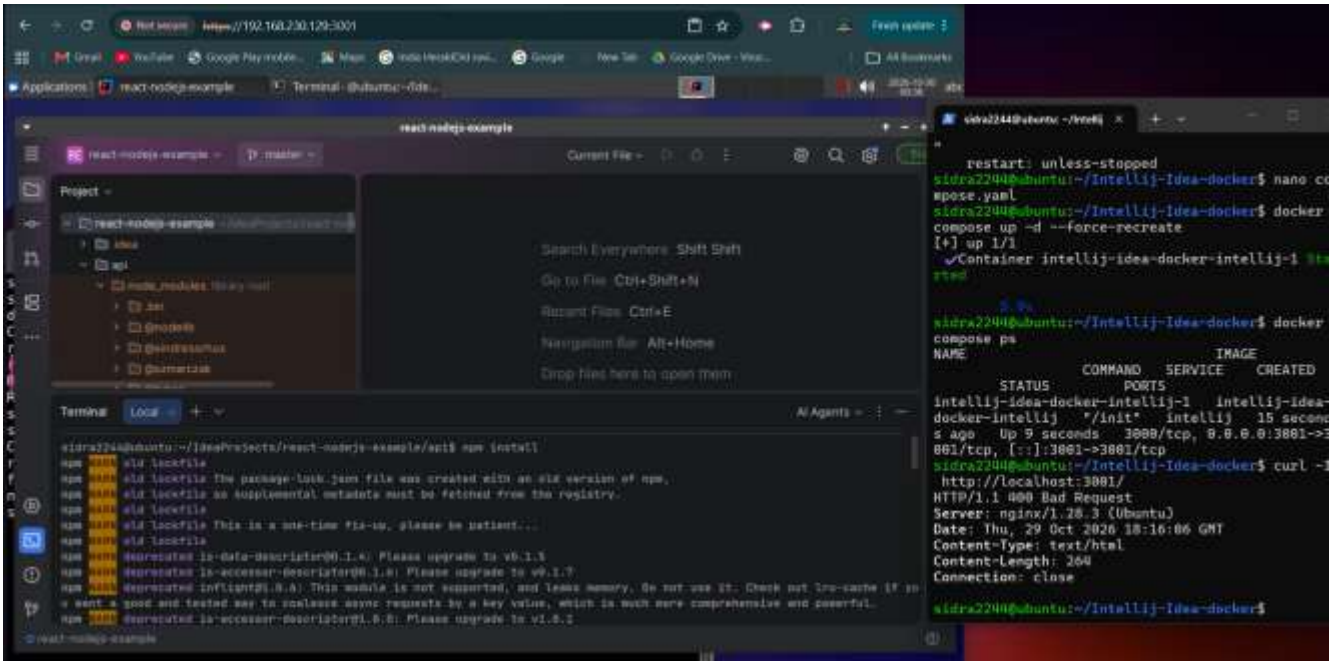


Figure caption: Completed npm install and api/node_modules.

Screenshot: react_api_build.png

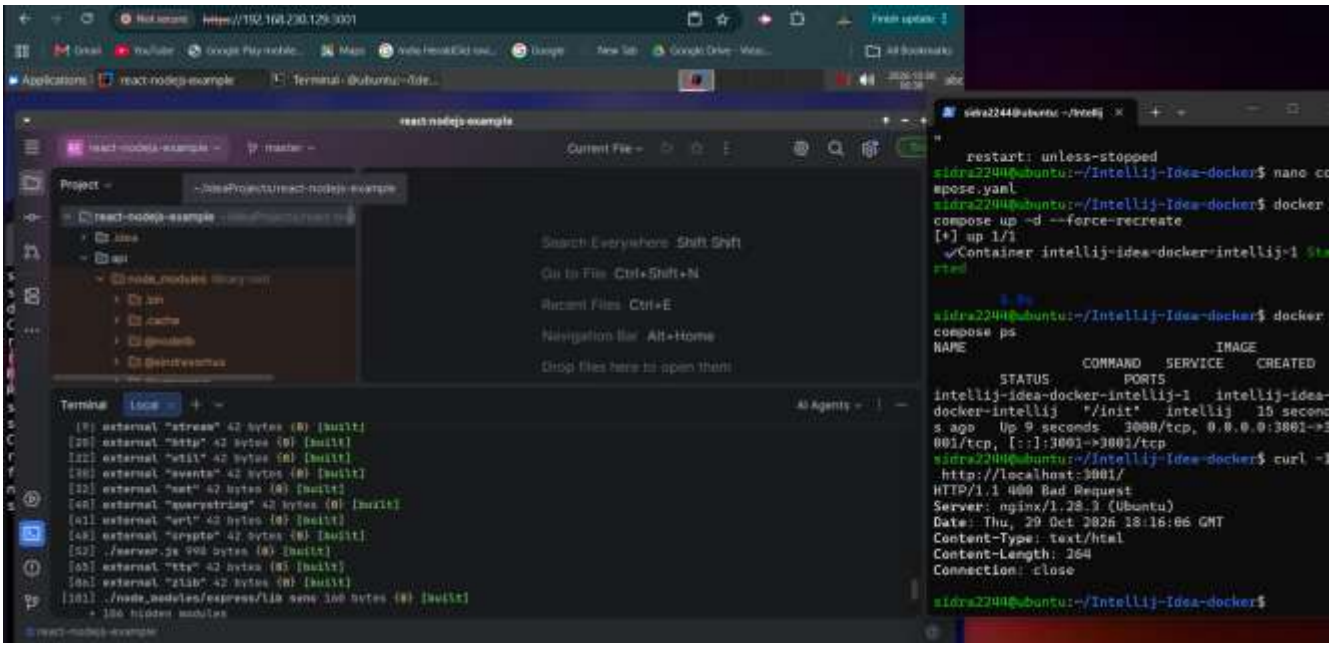


Figure caption: Successful webpack build and server.bundle.js.

Screenshot: react_api_bundle.png

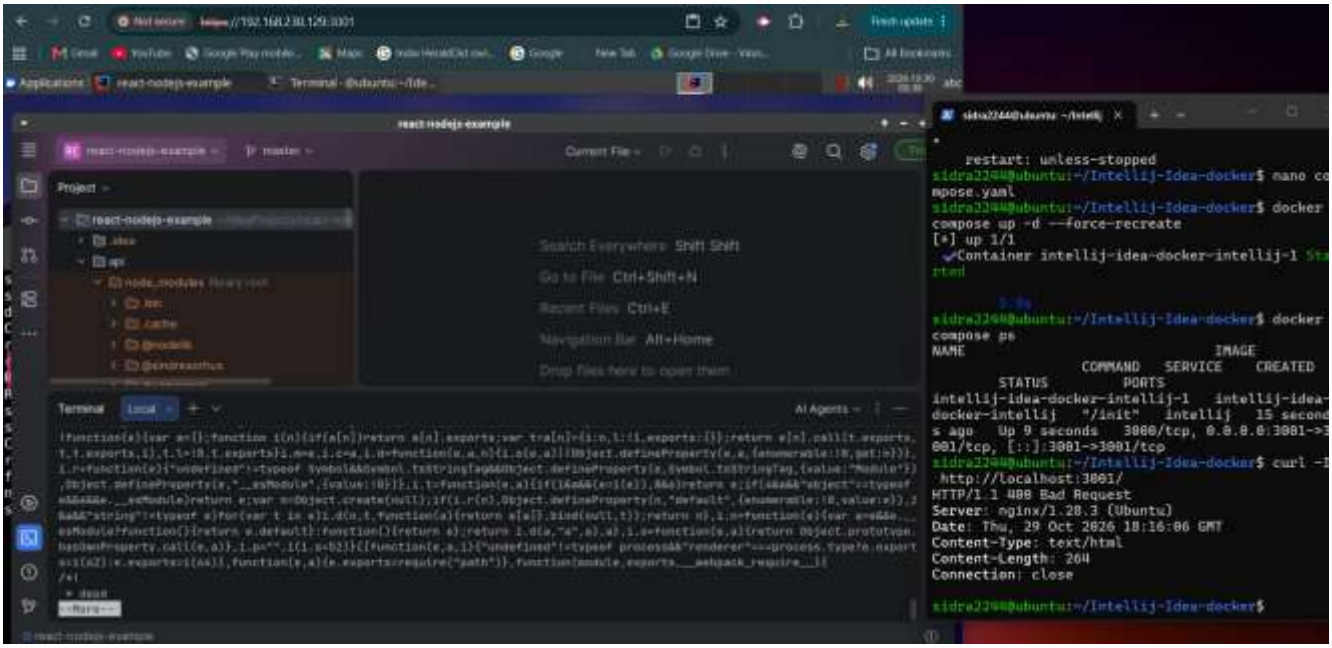


Figure caption: Bundle contents viewed in the terminal.

Task 6 — Build a Docker Image

Purpose

Inspect the existing Dockerfile, build the Node.js application image, and verify its tag.

Screenshot Evidence

Screenshot: docker_installed.png

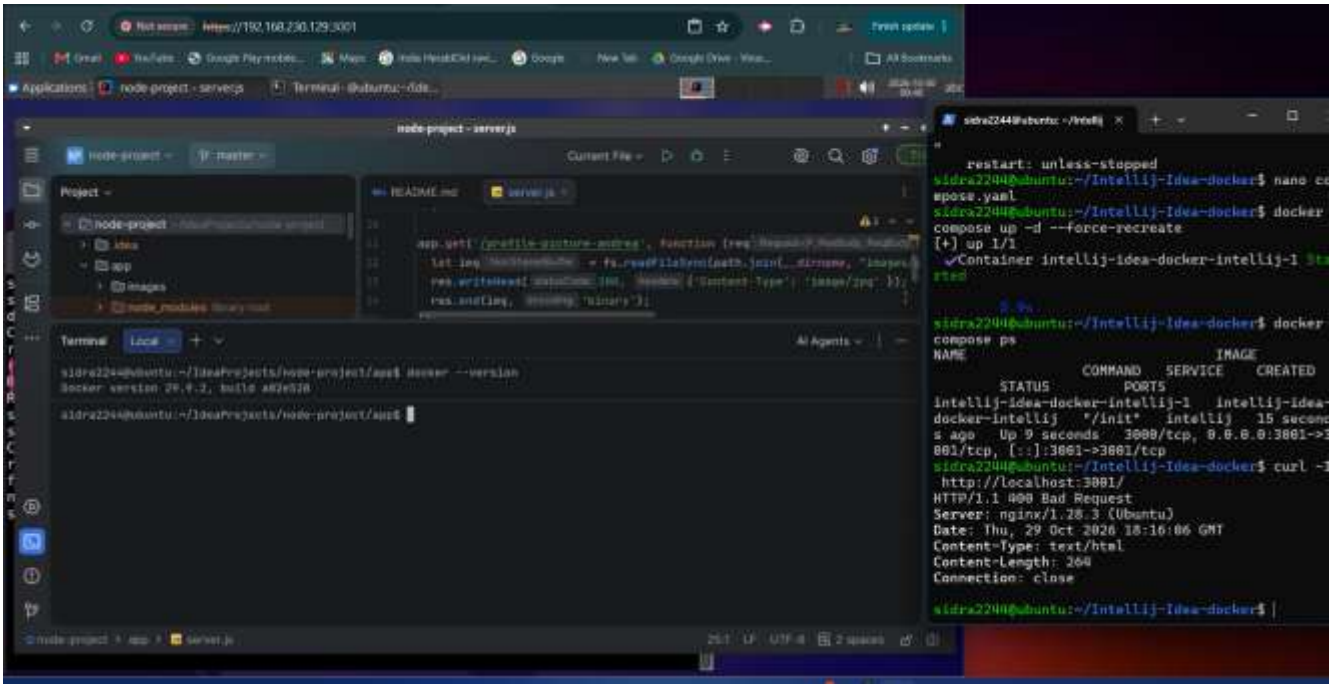


Figure caption: Docker version output.

Screenshot: dockerfile_exists.png

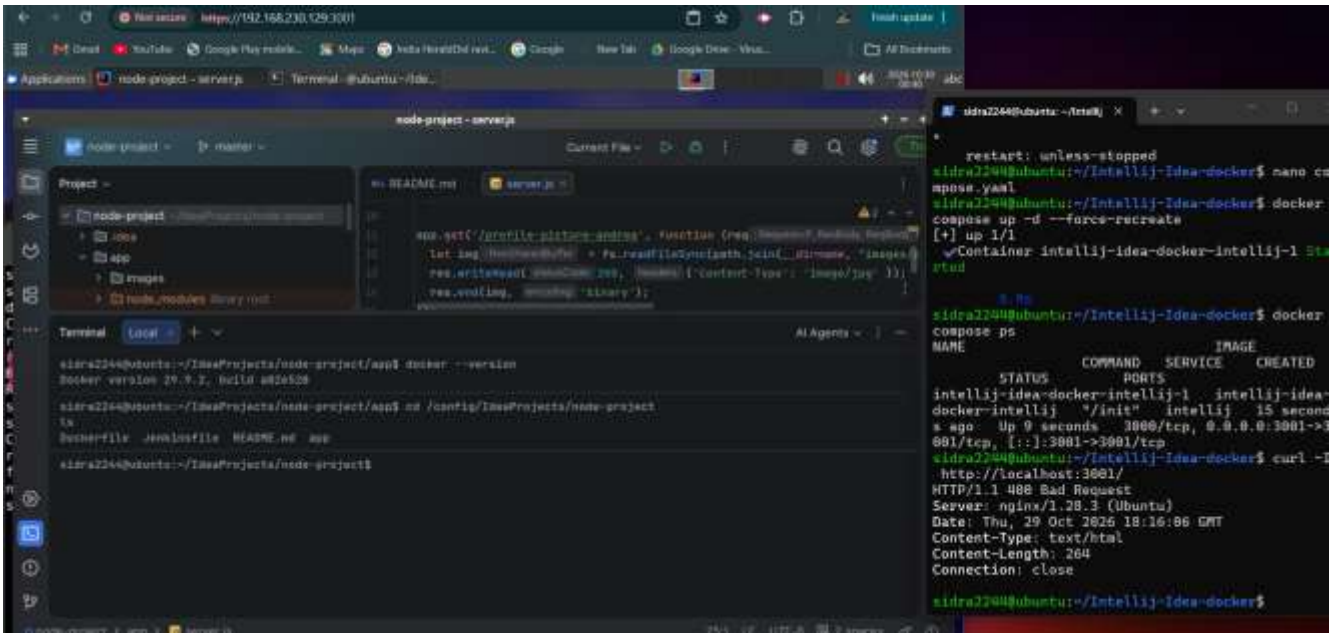


Figure caption: Repository listing containing Dockerfile.

Screenshot: docker_images_before.png

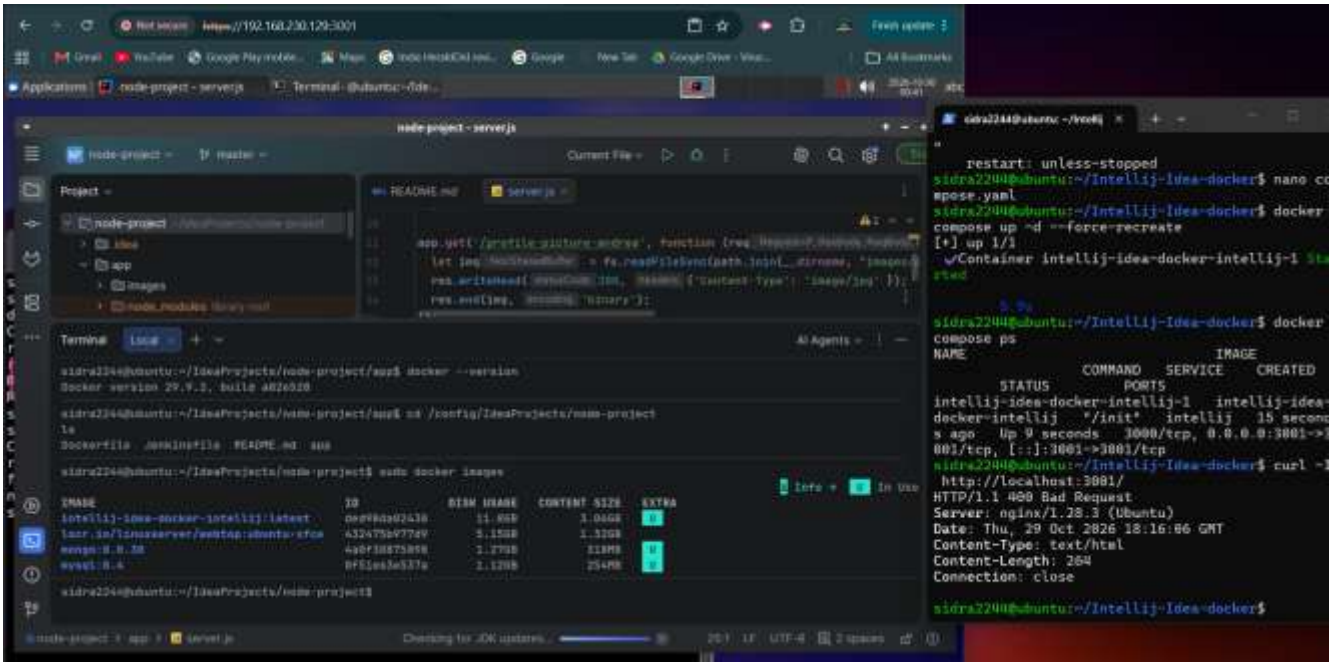


Figure caption: Docker image list before building.

Screenshot: docker_build.png

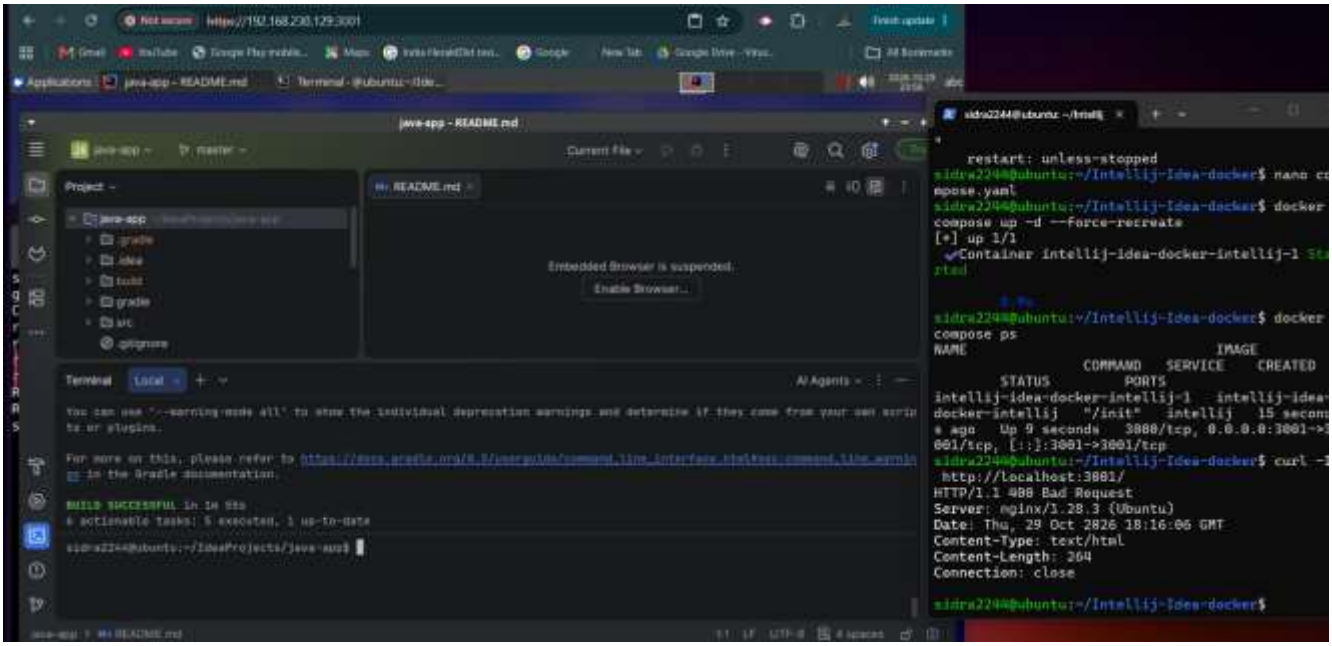


Figure caption: Successful Docker build and image tag.

Screenshot: docker_images_after.png

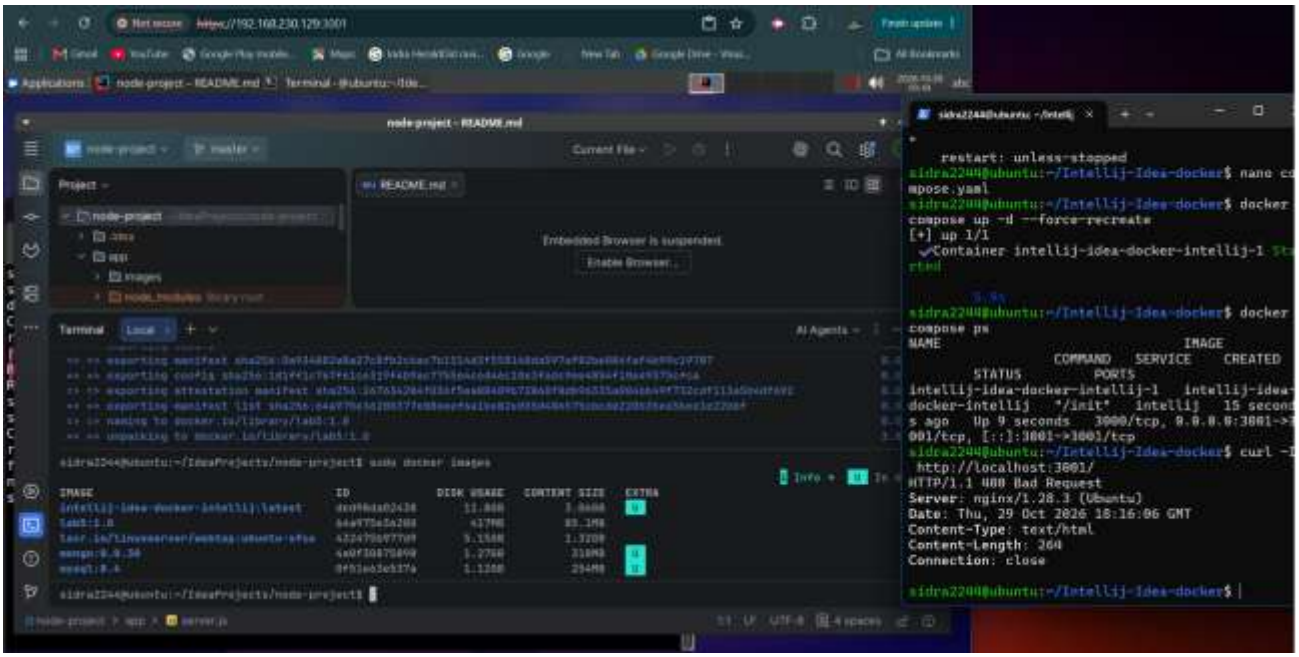


Figure caption: Image list showing lab5:1.0 and its image ID.

COMPLETED